Docker & Kubernetes with Minikube on Windows

Beginner to Advanced (Theory + Practical)

A Complete Learning Guide for Software Developers

Author: DevOps Training Institute
Target Audience: Developers familiar with JavaScript/React/Next.js and Python
Platform: Windows OS
Version: 2025 Edition

Table of Contents

Part 1: Docker Fundamentals

1. Introduction to Containerization
2. Docker Architecture & Installation
3. Docker Images & Containers
4. Dockerfile Mastery
5. Docker Networking
6. Docker Volumes & Data Persistence
7. Docker Compose
8. Advanced Docker Concepts

Part 2: Kubernetes Fundamentals

9. Introduction to Kubernetes
10. Kubernetes Architecture
11. Minikube Setup on Windows
12. Pods & Workloads
13. Services & Networking
14. ConfigMaps & Secrets
15. Volumes & Storage
16. Deployments & Rolling Updates

Part 3: Advanced Kubernetes

17. StatefulSets & DaemonSets

18. Ingress & Load Balancing
19. Resource Management
20. Health Checks & Monitoring
21. Helm Package Manager
22. CI/CD with Kubernetes

Part 4: Real-World Projects

23. Project 1: Containerizing a React App
24. Project 2: Microservices with Python
25. Project 3: Full-Stack Deployment

Appendices

- Appendix A: Command Reference
- Appendix B: Troubleshooting Guide
- Appendix C: Best Practices Checklist

Part 1: Docker Fundamentals

Chapter 1: Introduction to Containerization

1.1 What is Containerization?

Real-World Analogy:
Think of containers like shipping containers in the logistics industry. Just as a shipping container can hold various goods and be transported by truck, ship, or train without repacking, a software container packages an application with all its dependencies and can run consistently across different computing environments.

Definition:
Containerization is a lightweight form of virtualization that packages an application and its dependencies into a standardized unit called a container. Unlike virtual machines, containers share the host OS kernel, making them faster and more efficient.

1.2 The Problem Containers Solve

Traditional Development Challenges:

```
Developer's Machine          Staging Server          Production Server
├── Node.js v18              ├── Node.js v16         ├── Node.js v14
├── Python 3.11              ├── Python 3.9          ├── Python 3.8
├── Ubuntu 22.04             ├── CentOS 7            ├── RHEL 8
└── Works perfectly! ✓       └── Dependency errors ✗ └── Runtime crashes ✗
```

The infamous phrase: *"But it works on my machine!"*

1.3 Virtual Machines vs. Containers

```
Virtual Machines Architecture          Container Architecture

┌──────────────────────────┐           ┌──────────────────────────┐
│   App A   |   App B     |           │  App A  |  App B | App C|
├──────────┴──────────────┤           ├──────────┴────────┴──────┤
│   Guest OS |  Guest OS  |           │   Container Runtime      |
├──────────┴──────────────┤           ├──────────────────────────┤
│      Hypervisor         |           │      Host OS Kernel      |
├──────────────────────────┤           ├──────────────────────────┤
│   Infrastructure        |           │     Infrastructure       |
└──────────────────────────┘           └──────────────────────────┘

    Size: GBs each                      Size: MBs each
    Boot: Minutes                       Boot: Seconds
    Resource: Heavy                     Resource: Light
```

Comparison Table:

Feature	Virtual Machines	Containers
OS	Full OS per VM	Share host OS kernel
Size	Gigabytes	Megabytes
Startup	Minutes	Seconds
Performance	Limited	Native
Isolation	Complete	Process-level
Portability	Moderate	High
Resource Usage	Heavy	Lightweight

1.4 Benefits of Containerization

1. **Consistency**: "Write once, run anywhere"
2. **Isolation**: Applications don't interfere with each other
3. **Efficiency**: Lightweight and fast
4. **Scalability**: Easy to scale up or down
5. **Version Control**: Track changes to container images
6. **Microservices**: Perfect for distributed architectures

1.5 Container Ecosystem Overview

```
Container Ecosystem
│
├── Container Runtime
│   ├── Docker
│   ├── containerd
│   └── CRI-O
│
├── Orchestration
│   ├── Kubernetes
│   ├── Docker Swarm
│   └── Apache Mesos
│
├── Registry
│   ├── Docker Hub
│   ├── Amazon ECR
│   └── Google GCR
│
└── Development Tools
    ├── Docker Compose
    ├── Helm
    └── Skaffold
```

1.6 Use Cases for Containers

1. Microservices Architecture

```
Monolithic App              Microservices with Containers
┌──────────────┐           ┌────┐ ┌────┐ ┌────┐ ┌────┐
│              │           │Auth│ │User│ │Pay │ │Not │
│              │           │Svc │ │Svc │ │Svc │ │Svc │
│   All-in-One │    →      └────┘ └────┘ └────┘ └────┘
│              │             ↓      ↓      ↓      ↓
│              │           Each service in its own container
└──────────────┘           Scale independently, deploy independently
```

2. CI/CD Pipelines
3. Development Environments
4. Testing & QA
5. Cloud Migration

1.7 Container Lifecycle

```
Container Lifecycle

┌─────────┐
│  BUILD  │  docker build -t myapp .
└────┬────┘
     │
     ▼
┌─────────┐
│  PUSH   │  docker push myapp
└────┬────┘
     │
     ▼
┌─────────┐
│  PULL   │  docker pull myapp
└────┬────┘
     │
     ▼
┌─────────┐
│   RUN   │  docker run myapp
└────┬────┘
     │
     ▼
┌─────────┐
│  STOP   │  docker stop container_id
└────┬────┘
     │
     ▼
┌────────┐
│ REMOVE │  docker rm container_id
└────────┘
```

Lab 1.1: Understanding Your Development Environment

Objective: Prepare your Windows machine for containerization.

Prerequisites Check:

```
# Check Windows version (Requires Windows 10/11 Pro, Enterprise, or Education)
systeminfo | findstr /B /C:"OS Name" /C:"OS Version"

# Check if Hyper-V is available
systeminfo | findstr /C:"Hyper-V"

# Check PowerShell version (Recommended: 5.1 or higher)
$PSVersionTable.PSVersion
```

Expected Output:

```
OS Name:                   Microsoft Windows 11 Pro
OS Version:                10.0.22621 N/A Build 22621

PSVersion                      7.3.0
```

Exercise Questions:

1. Why do containers share the host OS kernel?
2. What are the advantages of containers over VMs?
3. Can you run Windows containers on Linux? Why or why not?

Chapter Summary

Key Takeaways:

- Containers package applications with their dependencies
- Containers are lighter and faster than virtual machines
- Docker is the most popular container runtime
- Containers enable consistent deployment across environments
- Containerization is essential for modern DevOps practices

Next Steps:
In the next chapter, we'll install Docker Desktop on Windows and explore Docker's architecture.

Interview Questions - Chapter 1

Q1: What is the main difference between containers and virtual machines?

Answer: Containers share the host OS kernel and virtualize at the operating system level, making them lightweight and fast to start (seconds). Virtual machines include a full OS and virtualize at the hardware

level through a hypervisor, making them heavier and slower to start (minutes). Containers use fewer resources but provide process-level isolation, while VMs provide complete isolation with their own OS.

Q2: Explain the "it works on my machine" problem and how containers solve it.

Answer: The problem occurs when code works in a developer's environment but fails in testing or production due to differences in dependencies, configurations, or OS versions. Containers solve this by packaging the application with all its dependencies, libraries, and configurations into a single unit that runs consistently across all environments.

Q3: What are the main benefits of using containers in a microservices architecture?

Answer:

- **Isolation**: Each microservice runs in its own container
- **Independent Scaling**: Scale services based on individual demand
- **Independent Deployment**: Update one service without affecting others
- **Technology Freedom**: Use different languages/frameworks per service
- **Resource Efficiency**: Lightweight compared to running separate VMs

Q4: When would you choose VMs over containers?

Answer:

- Need complete OS-level isolation
- Running applications requiring different OS kernels
- Legacy applications with specific OS dependencies
- Strong security requirements needing hardware-level isolation
- Applications requiring direct hardware access

Q5: What is a container registry and why is it important?

Answer: A container registry is a repository for storing and distributing container images (like Docker Hub, AWS ECR, or Google GCR). It's important because it:

- Enables sharing images across teams
- Provides version control for images
- Facilitates CI/CD pipelines
- Offers security scanning for vulnerabilities
- Supports private and public image repositories

Chapter 2: Docker Architecture & Installation

2.1 What is Docker?

Docker is an open-source platform that automates the deployment of applications inside containers. It provides tools to build, ship, and run containerized applications.

Docker Philosophy:

> "Build, Ship, and Run Any App, Anywhere"

2.2 Docker Architecture

```
Docker Architecture (Detailed)

┌──────────────────────────────────────────────────────────────┐
│                        Docker Client                         │
│  (CLI: docker build, docker pull, docker run, etc.)          │
└──────────────────────────┬───────────────────────────────────┘
                           │ REST API
                           ▼
┌──────────────────────────────────────────────────────────────┐
│                     Docker Daemon (dockerd)                  │
│  ┌──────────────────────────────────────────────────────┐    │
│  │              Image Management                         │    │
│  │  - Build images    - Pull images    - Push images    │    │
│  └──────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │            Container Management                       │    │
│  │  - Create  - Start  - Stop  - Remove                  │    │
│  └──────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │              Network Management                       │    │
│  │  - Bridge  - Host  - Overlay  - Custom               │    │
│  └──────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │              Volume Management                        │    │
│  │  - Local  - Network  - Named                          │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────┬───────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│                  Container Runtime (containerd)              │
│  ┌────────────────┐  ┌────────────────┐  ┌──────────────┐    │
│  │  Container 1   │  │  Container 2   │  │Container 3   │    │
│  │  ┌──────────┐  │  │  ┌──────────┐  │  │┌──────────┐  │    │
│  │  │ App + Deps│  │  │  │ App + Deps│  │  ||App+Deps│  │    │
│  │  └──────────┘  │  │  └──────────┘  │  │└──────────┘  │    │
│  └────────────────┘  └────────────────┘  └──────────────┘    │
└──────────────────────────┬───────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│                     Host Operating System                    │
```

```
│                    (Windows/Linux)                          │
└─────────────────────────────────────────────────────────────┘
```

Components Explained:

Component	Description	Responsibilities
Docker Client	CLI interface	Sends commands to daemon
Docker Daemon	Background service	Manages images, containers, networks
Docker Registry	Image repository	Stores and distributes images
Docker Images	Read-only templates	Blueprint for containers
Docker Containers	Running instances	Isolated runtime environments
Docker Volumes	Persistent storage	Data persistence beyond container lifecycle
Docker Networks	Communication layer	Enable container connectivity

2.3 Docker on Windows: Architecture Specifics

Docker Desktop for Windows uses different backends:

```
Windows Docker Architecture

Option 1: WSL 2 Backend (Recommended)
┌──────────────────────────────────────┐
│     Docker Desktop                   │
├──────────────────────────────────────┤
│     Docker Engine (Linux)            │
├──────────────────────────────────────┤
│   WSL 2 (Windows Subsystem Linux)    │
├──────────────────────────────────────┤
│       Windows 10/11 Kernel           │
└──────────────────────────────────────┘
  ✓ Better performance
  ✓ Full Linux compatibility
  ✓ Lower resource usage

Option 2: Hyper-V Backend (Legacy)
┌──────────────────────────────────────┐
│     Docker Desktop                   │
├──────────────────────────────────────┤
│   Docker Engine (Linux VM)           │
├──────────────────────────────────────┤
│          Hyper-V                     │
├──────────────────────────────────────┤
│       Windows 10/11 Kernel           │
└──────────────────────────────────────┘
  ⚠ Higher overhead
  ⚠ Slower file I/O
```

2.4 Installing Docker Desktop on Windows

Step 1: System Requirements

Minimum Requirements:

- Windows 10 64-bit: Pro, Enterprise, or Education (Build 19044 or higher)
- Windows 11 64-bit: Any edition
- Hardware virtualization enabled in BIOS
- 4GB RAM minimum (8GB recommended)
- 20GB free disk space

Step 2: Enable WSL 2 (Recommended)

```
# Open PowerShell as Administrator

# Enable WSL
wsl --install

# Set WSL 2 as default
wsl --set-default-version 2

# Install Ubuntu (or your preferred distro)
wsl --install -d Ubuntu

# Verify installation
wsl --list --verbose
```

Expected Output:

```
  NAME      STATE           VERSION
* Ubuntu    Running         2
```

Step 3: Enable Hyper-V (Alternative)

```
# Open PowerShell as Administrator

# Enable Hyper-V
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All

# Restart computer
Restart-Computer
```

Step 4: Download Docker Desktop

1. Visit: https://www.docker.com/products/docker-desktop
2. Download Docker Desktop for Windows
3. Run the installer (Docker Desktop Installer.exe)
4. Follow installation wizard
5. Choose WSL 2 backend when prompted
6. Restart computer if required

Step 5: Verify Installation

```
# Check Docker version
docker --version

# Check Docker Compose version
docker-compose --version

# Run test container
docker run hello-world

# Check Docker info
docker info
```

Expected Output:

```
Docker version 24.0.7, build afdd53b

Docker Compose version v2.23.3

Hello from Docker!
This message shows that your installation appears to be working correctly.

Server:
 Containers: 1
 Running: 0
 Paused: 0
 Stopped: 1
 Images: 1
 Server Version: 24.0.7
 Storage Driver: overlay2
```

2.5 Docker Desktop Interface

```
Docker Desktop Components


┌──────────────────────────────────────────────────────────┐
│  Docker Desktop GUI                                      │
├──────────────────────────────────────────────────────────┤
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐    │
│  │ Containers   │  │   Images     │  │   Volumes    │    │
│  │              │  │              │  │              │    │
│  │ - Running    │  │ - Local      │  │ - Named      │    │
│  │ - Stopped    │  │ - Hub        │  │ - Anonymous  │    │
│  └──────────────┘  └──────────────┘  └──────────────┘    │
│  ┌──────────────────────────────────────────────────┐    │
│  │            Settings & Configuration              |    |
│  |  - Resources  - Docker Engine  - Extensions      |    |
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘
```

2.6 Configuring Docker Desktop

Resource Allocation:

```
# Open Docker Desktop → Settings → Resources

# Recommended settings:
# CPUs: 4 (minimum 2)
# Memory: 8GB (minimum 4GB)
# Swap: 2GB
# Disk image size: 60GB
```

Docker Engine Configuration:

Navigate to Settings → Docker Engine, modify daemon.json:

```
{
  "builder": {
    "gc": {
      "defaultKeepStorage": "20GB",
      "enabled": true
    }
  },
  "experimental": false,
  "features": {
    "buildkit": true
  },
  "registry-mirrors": []
}
```

2.7 Docker CLI Basics

Command Structure:

```
docker [OPTIONS] COMMAND [ARG...]

Example:
docker run -d -p 80:80 nginx
|      |  |  |      |
|      |  |  |      └─ Image name
|      |  |  └──────── Port mapping
|      |  └─────────── Detached mode
|      └──────────────── Run command
└─────────────────────── Docker CLI
```

Essential Commands:

```
# System Information
docker version              # Show Docker version
docker info                # Display system-wide information
docker --help              # List all commands

# Image Management
docker images              # List images
docker pull <image>        # Download image
docker rmi <image>         # Remove image
docker build -t <name> .   # Build image from Dockerfile

# Container Management
docker ps                  # List running containers
docker ps -a               # List all containers
docker run <image>         # Create and start container
docker start <container>   # Start stopped container
docker stop <container>    # Stop running container
docker rm <container>      # Remove container

# Logs & Debugging
docker logs <container>    # View container logs
docker exec -it <container> bash  # Access container shell
docker inspect <container> # Detailed container info
```

Lab 2.1: Docker Installation and Verification

Objective: Install Docker Desktop and verify it's working correctly.

Task 1: Pre-Installation Check

```
# Check Windows version
winver

# Check if virtualization is enabled
systeminfo | findstr /C:"Hyper-V"

# Check WSL version
wsl --list --verbose
```

Task 2: Install Docker Desktop

Follow the installation steps in section 2.4.

Task 3: Verify Installation

```
# 1. Check Docker version
docker --version

# 2. Run hello-world container
docker run hello-world

# 3. Check running containers
docker ps

# 4. Check all containers (including stopped)
docker ps -a

# 5. View downloaded images
docker images

# 6. Check Docker system info
docker info | findstr "Server Version"
docker info | findstr "Storage Driver"
docker info | findstr "Operating System"
```

Task 4: Explore Docker Desktop GUI

1. Open Docker Desktop from Start Menu
2. Navigate to Containers/Apps tab
3. Navigate to Images tab
4. Navigate to Volumes tab
5. Check Settings → Resources

Expected Results:

- Docker version 24.x or higher
- hello-world container runs successfully
- Docker Desktop GUI is accessible
- No error messages in PowerShell

Lab 2.2: Running Your First Container

Objective: Understand basic container operations.

Task 1: Run an Nginx Web Server

```
# Pull and run nginx container
docker run -d -p 8080:80 --name my-nginx nginx

# Breakdown:
# -d              = detached mode (runs in background)
# -p 8080:80      = map host port 8080 to container port 80
# --name my-nginx = name the container "my-nginx"
# nginx           = image name
```

Task 2: Verify Container is Running

```
# List running containers
docker ps

# Expected output:
# CONTAINER ID   IMAGE   COMMAND                  PORTS                   NAMES
# abc123def456   nginx   "/docker-entrypoint.…"  0.0.0.0:8080->80/tcp   my-nginx
```

Task 3: Access the Web Server

Open browser and navigate to: http://localhost:8080

You should see: "Welcome to nginx!"

Task 4: View Container Logs

```
# View logs
docker logs my-nginx

# Follow logs in real-time
docker logs -f my-nginx

# Press Ctrl+C to stop following
```

Task 5: Execute Commands Inside Container

```
# Access container shell
docker exec -it my-nginx bash

# Inside container, run:
ls /usr/share/nginx/html/
cat /usr/share/nginx/html/index.html
exit
```

Task 6: Stop and Remove Container

```
# Stop container
docker stop my-nginx

# Verify it's stopped
docker ps -a

# Remove container
docker rm my-nginx

# Verify removal
docker ps -a
```

Challenge Exercise:

Run multiple instances of nginx on different ports:

```
docker run -d -p 8081:80 --name nginx-1 nginx
docker run -d -p 8082:80 --name nginx-2 nginx
docker run -d -p 8083:80 --name nginx-3 nginx

# Access each in browser:
# http://localhost:8081
# http://localhost:8082
# http://localhost:8083

# Clean up
docker stop nginx-1 nginx-2 nginx-3
docker rm nginx-1 nginx-2 nginx-3
```

Troubleshooting Common Installation Issues

Issue 1: "WSL 2 installation is incomplete"

```
# Solution: Update WSL kernel
wsl --update

# Set WSL 2 as default
wsl --set-default-version 2

# Restart Docker Desktop
```

Issue 2: "Docker daemon is not running"

```
# Solution 1: Restart Docker Desktop
# Close Docker Desktop completely
# Open Docker Desktop from Start Menu

# Solution 2: Check Windows Services
Get-Service | Where-Object {$_.Name -like "*docker*"}

# Restart service if needed
Restart-Service -Name com.docker.service
```

Issue 3: "Hyper-V is not enabled"

```
# Solution: Enable Hyper-V
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All

# Restart computer
Restart-Computer
```

Issue 4: Port conflicts

```
# Check what's using a port
netstat -ano | findstr :8080

# Kill process if needed
taskkill /PID <process_id> /F
```

Best Practices - Docker Installation

1. **Use WSL 2 Backend**
    - Better performance than Hyper-V
    - Lower resource consumption
    - Native Linux compatibility
2. **Resource Allocation**
    - Allocate at least 4GB RAM for Docker
    - Reserve 2+ CPU cores
    - Ensure sufficient disk space (20GB minimum)
3. **Security**
    - Keep Docker Desktop updated
    - Only pull images from trusted registries
    - Review image permissions
4. **Performance**
    - Enable BuildKit for faster builds
    - Use .dockerignore files
    - Clean up unused images regularly
5. **Development Workflow**
    - Use bind mounts for live code updates
    - Leverage Docker Compose for multi-container apps
    - Tag images properly for version control

Chapter Summary

Key Takeaways:

- Docker consists of Client, Daemon, Registry, Images, and Containers
- Docker Desktop for Windows uses WSL 2 or Hyper-V backend
- WSL 2 backend offers better performance
- Basic commands: run, ps, images, logs, exec, stop, rm
- Docker Desktop provides GUI for container management
- Port mapping connects host to container ports

Command Cheat Sheet:

```
# Installation verification
docker --version
docker run hello-world

# Container lifecycle
docker run <image>         # Create and start
docker stop <container>    # Stop
docker start <container>   # Start
docker rm <container>      # Remove

# Information
docker ps                  # Running containers
docker ps -a               # All containers
docker images              # List images
docker logs <container>    # View logs
```

Interview Questions - Chapter 2

Q1: Explain the difference between Docker images and containers.

Answer: A Docker image is a read-only template that contains the application code, runtime, libraries, and dependencies. It's like a blueprint or snapshot. A container is a running instance of an image. When you execute `docker run`, Docker creates a container from an image. Multiple containers can be created from the same image, each running independently.

Analogy: Image is like a class in OOP, container is like an object instance of that class.

Q2: What is the Docker daemon and what role does it play?

Answer: The Docker daemon (dockerd) is a background service that manages Docker objects like images, containers, networks, and volumes. It listens for Docker API requests from the Docker client and processes them. The daemon is responsible for:

- Building images from Dockerfiles
- Pulling images from registries
- Creating and managing containers
- Managing networks and volumes
- Communicating with container runtime (containerd)

Q3: Why is WSL 2 recommended over Hyper-V for Docker Desktop on Windows?

Answer:

- **Performance**: WSL 2 has faster file I/O operations

- **Resource Efficiency**: Lower memory and CPU overhead
- **Startup Time**: Containers start faster
- **Full Compatibility**: Native Linux kernel provides better compatibility
- **Integration**: Better integration with Windows file system
- **Updates**: Receives faster kernel updates

Q4: What happens when you run `docker run -d -p 8080:80 nginx`? Explain each part.

Answer:

- `docker run`: Creates and starts a new container
- `-d`: Detached mode - runs container in background
- `-p 8080:80`: Port mapping - maps host port 8080 to container port 80
- `nginx`: Image name to use

Process:

1. Docker checks if nginx image exists locally
2. If not, pulls it from Docker Hub
3. Creates a new container from the image
4. Maps specified ports
5. Starts container in background
6. Returns container ID

Q5: How do you troubleshoot a container that won't start?

Answer:

```
# 1. Check container status
docker ps -a

# 2. View logs
docker logs <container-name>

# 3. Inspect container configuration
docker inspect <container-name>

# 4. Check if port is already in use
netstat -ano | findstr :<port>

# 5. Try running in foreground (remove -d) to see errors
docker run <image>

# 6. Check resource availability
docker info
```

Common issues: port conflicts, insufficient resources, incorrect image, misconfigured environment variables.

Chapter 3: Docker Images & Containers

3.1 Understanding Docker Images

Definition:
A Docker image is a lightweight, standalone, executable package that includes everything needed to run software: code, runtime, libraries, environment variables, and configuration files.

Image Layers:

```
Docker Image Layer Architecture


┌──────────────────────────────────────┐
│  Application Layer (Your Code)       │  ← Read/Write (Container Layer)
├──────────────────────────────────────┤
│  Dependencies Layer (npm install)    │  ← Read-Only
├──────────────────────────────────────┤
│  Runtime Layer (Node.js)             │  ← Read-Only
├──────────────────────────────────────┤
│  Base OS Layer (Ubuntu/Alpine)       │  ← Read-Only
└──────────────────────────────────────┘


Each layer is cached and reusable!
```

How Layers Work:

```
Example: Building a Node.js App

Layer 1: FROM node:18-alpine       (50 MB)
         ↓
Layer 2: COPY package.json .        (1 KB)
         ↓
Layer 3: RUN npm install            (100 MB)
         ↓
Layer 4: COPY . .                   (5 MB)
         ↓
Layer 5: CMD ["node", "app.js"]     (0 MB - metadata)

Total Image Size: ~155 MB
Layers 1-3 are cached and reused across builds if unchanged!
```

3.2 Working with Docker Images

Image Naming Convention:

```
registry/repository:tag

Examples:
nginx:latest                    # Official nginx, latest version
docker.io/nginx:1.25.3         # Full specification
myusername/myapp:v1.0.0        # Personal image
gcr.io/myproject/api:prod      # Google Container Registry
```

Components:

- **Registry**: Where images are stored (docker.io by default)
- **Repository**: Collection of related images (nginx, mysql, etc.)
- **Tag**: Version identifier (latest, v1.0.0, alpine, etc.)

3.3 Essential Image Commands

```
# ========== SEARCHING IMAGES ==========
# Search Docker Hub
docker search nginx

# Search with filters
docker search --filter stars=1000 python

# ========== PULLING IMAGES ==========
# Pull latest version
docker pull nginx

# Pull specific version
docker pull nginx:1.25.3

# Pull from specific registry
docker pull mcr.microsoft.com/dotnet/aspnet:7.0

# ========== LISTING IMAGES ==========
# List all images
docker images

# List with full details
docker images --no-trunc

# List specific image
docker images nginx

# List image IDs only
docker images -q

# ========== INSPECTING IMAGES ==========
# Detailed information
docker inspect nginx

# View image history (layers)
docker history nginx

# View image layers
docker image inspect nginx --format='{{.RootFS.Layers}}'

# ========== REMOVING IMAGES ==========
# Remove single image
```

```
docker rmi nginx

# Remove by ID
docker rmi abc123def456

# Force remove (even if container exists)
docker rmi -f nginx

# Remove multiple images
docker rmi nginx redis mongodb

# Remove all unused images
docker image prune

# Remove all images
docker rmi $(docker images -q)

# ========== TAGGING IMAGES ==========
# Create new tag for existing image
docker tag nginx:latest myrepo/nginx:v1

# Tag with custom registry
docker tag myapp localhost:5000/myapp:latest

# ========== SAVING & LOADING IMAGES ==========
# Save image to tar file
docker save nginx -o nginx.tar

# Load image from tar file
docker load -i nginx.tar

# Export to compressed file
docker save nginx | gzip > nginx.tar.gz
```

3.4 Understanding Containers

Container Lifecycle:

```
Container States

      ┌──────────┐
      │ CREATED  │  ← docker create
      └──────┬───┘
             │
             ▼
      ┌──────────┐
      │ RUNNING  │  ← docker start / docker run
      └──┬────┬──┘
         │    │
    pause │    │ unpause
         │    │
      ┌──▼────▼──┐
      │  PAUSED  │  ← docker pause
      └──────────┘

         │
         │ stop
         ▼
      ┌──────────┐
      │ STOPPED  │  ← docker stop
      └──────┬───┘
             │
             │ remove
             ▼
      ┌──────────┐
      │ REMOVED  │  ← docker rm
      └──────────┘
```

3.5 Container Operations

Running Containers - Complete Guide:

```
# ========== BASIC RUN ==========
# Simple run (foreground)
docker run nginx

# Detached mode (background)
docker run -d nginx

# With custom name
docker run -d --name webserver nginx

# ========== PORT MAPPING ==========
# Map single port
docker run -d -p 8080:80 nginx

# Map multiple ports
docker run -d -p 8080:80 -p 8443:443 nginx

# Map all exposed ports to random host ports
docker run -d -P nginx

# Map to specific host IP
docker run -d -p 127.0.0.1:8080:80 nginx

# ========== ENVIRONMENT VARIABLES ==========
# Set single variable
docker run -d -e "NODE_ENV=production" node-app

# Set multiple variables
docker run -d \
  -e "DB_HOST=localhost" \
  -e "DB_PORT=5432" \
  -e "DB_NAME=mydb" \
  postgres

# Load from file
docker run -d --env-file ./config.env node-app

# ========== VOLUME MOUNTING ==========
# Bind mount (Windows paths)
docker run -d -v C:\Users\dev\code:/app node-app

# Named volume
```

```
docker run -d -v mydata:/data postgres

# Read-only mount
docker run -d -v C:\data:/app/data:ro nginx

# ========== RESOURCE LIMITS ==========
# Limit memory
docker run -d -m 512m nginx

# Limit CPU
docker run -d --cpus="1.5" nginx

# Combined limits
docker run -d -m 512m --cpus="1" --name limited-nginx nginx

# ========== NETWORK OPTIONS ==========
# Use host network
docker run -d --network host nginx

# Use specific network
docker run -d --network my-network nginx

# Create and use network
docker network create mynet
docker run -d --network mynet --name web nginx

# ========== RESTART POLICIES ==========
# Always restart
docker run -d --restart always nginx

# Restart on failure only
docker run -d --restart on-failure nginx

# Restart unless stopped
docker run -d --restart unless-stopped nginx

# No restart
docker run -d --restart no nginx

# ========== INTERACTIVE MODE ==========
# Interactive terminal
docker run -it ubuntu bash
```

```
# Interactive with cleanup
docker run -it --rm ubuntu bash

# Execute command and exit
docker run ubuntu ls -la

# ========== CLEANUP ==========
# Auto-remove after stop
docker run --rm nginx

# Run and remove immediately after execution
docker run --rm ubuntu echo "Hello World"
```

3.6 Managing Running Containers

```
# ========== LISTING CONTAINERS ==========
# List running containers
docker ps

# List all containers (including stopped)
docker ps -a

# List with specific format
docker ps --format "table {{.ID}}\t{{.Names}}\t{{.Status}}"

# Show last created container
docker ps -l

# Show n last created containers
docker ps -n 3

# Show only container IDs
docker ps -q

# Show size
docker ps -s

# ========== STARTING & STOPPING ==========
# Start stopped container
docker start webserver

# Start multiple containers
docker start web1 web2 web3

# Stop running container (graceful shutdown, 10s timeout)
docker stop webserver

# Force stop (immediate)
docker kill webserver

# Restart container
docker restart webserver

# Pause container (freeze)
docker pause webserver

# Unpause container
```

```
docker unpause webserver

# ========== CONTAINER INFORMATION ==========
# View logs
docker logs webserver

# Follow logs (real-time)
docker logs -f webserver

# Show last 100 lines
docker logs --tail 100 webserver

# Show logs with timestamps
docker logs -t webserver

# View container stats (CPU, memory, network, disk)
docker stats webserver

# Real-time stats for all containers
docker stats

# View processes running in container
docker top webserver

# View detailed container info
docker inspect webserver

# View specific field from inspect
docker inspect --format='{{.NetworkSettings.IPAddress}}' webserver

# ========== EXECUTING COMMANDS ==========
# Execute command in running container
docker exec webserver ls -la

# Interactive shell
docker exec -it webserver bash

# Execute as specific user
docker exec -u root webserver whoami

# Execute with environment variable
docker exec -e "VAR=value" webserver env
```

```
# ========== COPYING FILES ==========
# Copy from container to host
docker cp webserver:/app/logs.txt ./logs.txt

# Copy from host to container
docker cp ./config.json webserver:/app/config.json

# Copy directory
docker cp webserver:/app/data ./backup-data

# ========== CONTAINER MODIFICATION ==========
# Rename container
docker rename old-name new-name

# Update container configuration
docker update --memory 512m webserver

# Update restart policy
docker update --restart unless-stopped webserver

# ========== REMOVING CONTAINERS ==========
# Remove stopped container
docker rm webserver

# Force remove running container
docker rm -f webserver

# Remove multiple containers
docker rm web1 web2 web3

# Remove all stopped containers
docker container prune

# Remove all containers (running and stopped)
docker rm -f $(docker ps -aq)

# ========== CONTAINER EXPORT/IMPORT ==========
# Export container filesystem to tar
docker export webserver > webserver.tar

# Import tar as image
docker import webserver.tar myimage:v1
```

```
# Commit container changes to new image
docker commit webserver mynewimage:v1

# Commit with message and author
docker commit -m "Added config" -a "John Doe" webserver myimage:v2
```

3.7 Docker Image Layers Deep Dive

Layer Caching Mechanism:

```
# Example: Understanding layer caching

# Layer 1 - Cached if Node.js version unchanged
FROM node:18-alpine

# Layer 2 - Cached if working directory same
WORKDIR /app

# Layer 3 - Cached if package.json unchanged
COPY package.json package-lock.json ./

# Layer 4 - Cached if package files unchanged (expensive!)
RUN npm install

# Layer 5 - Changes frequently, NOT cached
COPY . .

# Layer 6 - Metadata only
CMD ["node", "server.js"]
```

Optimization Strategy:

```
Bad Practice (Slow rebuilds):
COPY . .                    ← Everything copied first
RUN npm install             ← Runs every time!
CMD ["node", "app.js"]

Good Practice (Fast rebuilds):
COPY package*.json ./       ← Only dependency files
RUN npm install             ← Cached if dependencies unchanged
COPY . .                    ← Code copied last
CMD ["node", "app.js"]
```

Lab 3.1: Working with Images

Objective: Master image operations and understand layering.

Task 1: Explore Docker Hub

```
# Search for Python images
docker search python

# Search with star filter
docker search --filter stars=100 python

# Pull official Python image
docker pull python:3.11-slim

# Pull specific version
docker pull python:3.11-alpine

# Pull multiple images
docker pull node:18
docker pull redis:7
docker pull postgres:15
```

Task 2: Inspect Images

```
# List all images
docker images

# Inspect Python image
docker inspect python:3.11-slim

# View image history (layers)
docker history python:3.11-slim

# View specific information
docker inspect python:3.11-slim --format='{{.Size}}'
docker inspect python:3.11-slim --format='{{.Architecture}}'
```

Task 3: Tag and Save Images

```
# Create custom tag
docker tag python:3.11-slim mypython:dev

# Create multiple tags
docker tag python:3.11-slim mypython:v1
docker tag python:3.11-slim mypython:latest

# Save image to file
docker save python:3.11-slim -o python-slim.tar

# Check file size
Get-Item python-slim.tar | Select-Object Name, Length

# Load image from file
docker load -i python-slim.tar
```

Task 4: Image Cleanup

```
# Remove specific image
docker rmi node:18

# Remove by ID
docker rmi $(docker images -q python:3.11-alpine)

# Remove unused images
docker image prune

# Remove all images (careful!)
docker rmi $(docker images -q)
```

Lab 3.2: Container Lifecycle Management

Objective: Understand container states and operations.

Task 1: Create and Manage Containers

```
# Create container without starting
docker create --name myredis redis

# Verify created state
docker ps -a

# Start the container
docker start myredis

# Verify running state
docker ps

# Pause the container
docker pause myredis

# Check stats (should show paused)
docker stats --no-stream myredis

# Unpause
docker unpause myredis

# Stop gracefully
docker stop myredis

# Force stop
docker kill myredis

# Remove
docker rm myredis
```

Task 2: Running Containers with Options

```
# Run with port mapping
docker run -d \
  --name web \
  -p 8080:80 \
  nginx

# Run with environment variables
docker run -d \
  --name postgres-db \
  -e POSTGRES_PASSWORD=mysecret \
  -e POSTGRES_USER=admin \
  -e POSTGRES_DB=testdb \
  postgres:15

# Run with resource limits
docker run -d \
  --name limited-nginx \
  -m 256m \
  --cpus="0.5" \
  -p 8081:80 \
  nginx

# Run with restart policy
docker run -d \
  --name persistent-redis \
  --restart unless-stopped \
  redis

# Run with volume mount
docker run -d \
  --name data-container \
  -v mydata:/data \
  -e POSTGRES_PASSWORD=secret \
  postgres
```

Task 3: Interacting with Containers

```
# View logs
docker logs web

# Follow logs in real-time
docker logs -f web

# View last 50 lines
docker logs --tail 50 web

# Execute command
docker exec web ls -la /usr/share/nginx/html

# Interactive shell
docker exec -it postgres-db psql -U admin -d testdb

# Inside PostgreSQL shell, run:
# \l          -- List databases
# \c testdb   -- Connect to database
# CREATE TABLE test (id INT, name VARCHAR(50));
# \q          -- Quit

# Check container stats
docker stats --no-stream

# View processes
docker top web
```

Task 4: Container Information

```
# Inspect container
docker inspect web

# Get specific information
docker inspect --format='{{.NetworkSettings.IPAddress}}' web
docker inspect --format='{{.State.Status}}' web
docker inspect --format='{{.RestartCount}}' web

# View port mappings
docker port web

# Check resource usage
docker stats --no-stream web
```

Task 5: Copy Files

```
# Create local HTML file
@"
<!DOCTYPE html>
<html>
<body>
<h1>Custom Page</h1>
<p>This is my custom nginx page!</p>
</body>
</html>
"@ | Out-File -FilePath custom.html

# Copy to container
docker cp custom.html web:/usr/share/nginx/html/index.html

# Verify in browser: http://localhost:8080

# Copy from container
docker cp web:/etc/nginx/nginx.conf ./nginx-backup.conf
```

Task 6: Container Cleanup

```
# Stop all running containers
docker stop $(docker ps -q)

# Remove all containers
docker rm $(docker ps -aq)

# Prune stopped containers
docker container prune

# Complete cleanup (containers, networks, images, cache)
docker system prune -a
```

Lab 3.3: Advanced Container Scenarios

Objective: Apply container concepts in real scenarios.

Scenario 1: Multi-Tier Application

```
# Create custom network
docker network create app-network

# Run database
docker run -d \
  --name postgres-db \
  --network app-network \
  -e POSTGRES_PASSWORD=secret123 \
  -e POSTGRES_DB=appdb \
  postgres:15

# Run Redis cache
docker run -d \
  --name redis-cache \
  --network app-network \
  redis:7

# Run web application (simulated with nginx)
docker run -d \
  --name web-app \
  --network app-network \
  -p 8080:80 \
  nginx

# Verify connectivity
docker exec web-app ping -c 3 postgres-db
docker exec web-app ping -c 3 redis-cache

# View network details
docker network inspect app-network
```

Scenario 2: Container with Persistent Data

```
# Create named volume
docker volume create pgdata

# Run PostgreSQL with volume
docker run -d \
  --name postgres-persistent \
  -v pgdata:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=secret \
  postgres:15

# Add data
docker exec -it postgres-persistent psql -U postgres -c "CREATE TABLE users (id
INT, name VARCHAR(50));"
docker exec -it postgres-persistent psql -U postgres -c "INSERT INTO users VALUES
(1, 'John Doe');"

# Stop and remove container
docker stop postgres-persistent
docker rm postgres-persistent

# Create new container with same volume
docker run -d \
  --name postgres-restored \
  -v pgdata:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=secret \
  postgres:15

# Verify data persists
docker exec postgres-restored psql -U postgres -c "SELECT * FROM users;"
```

Scenario 3: Development Environment

```
# Create project directory
New-Item -ItemType Directory -Path C:\Projects\myapp
Set-Location C:\Projects\myapp

# Create simple Node.js app
@"
const http = require('http');
const server = http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Hello from Docker Container!\\n');
});
server.listen(3000, '0.0.0.0');
console.log('Server running on port 3000');
"@ | Out-File -FilePath server.js

# Run with bind mount for live updates
docker run -d \
  --name dev-server \
  -p 3000:3000 \
  -v C:\Projects\myapp:/app \
  -w /app \
  node:18 \
  node server.js

# Test
# Open browser: http://localhost:3000

# Modify server.js locally and restart container
docker restart dev-server
```

Troubleshooting Common Container Issues

Issue 1: Container exits immediately

```
# Check logs
docker logs <container-name>

# Run in foreground to see errors
docker run <image>

# Check if command is valid
docker inspect <container-name> --format='{{.Config.Cmd}}'
```

Issue 2: Cannot access container port

```
# Verify port mapping
docker port <container-name>

# Check if container is running
docker ps

# Verify firewall isn't blocking
# Windows Defender Firewall → Allow an app

# Check if port is already in use
netstat -ano | findstr :<port>
```

Issue 3: Container out of memory

```
# Check memory usage
docker stats <container-name>

# Increase memory limit
docker update --memory 1g <container-name>

# Or restart with higher limit
docker run -m 1g <image>
```

Issue 4: Cannot connect to other containers

```
# Check if containers are on same network
docker network inspect <network-name>

# Connect container to network
docker network connect <network> <container>

# Test connectivity
docker exec <container1> ping <container2>
```

Best Practices - Images & Containers

Image Best Practices:

1. **Use Official Images**

```
# Good
docker pull nginx:1.25.3

# Avoid unverified sources
```

2. **Pin Versions**

```
# Good - specific version
docker pull node:18.19.0-alpine

# Avoid - unpredictable updates
docker pull node:latest
```

3. **Choose Lightweight Base Images**

```
# Ubuntu: ~77MB
docker pull ubuntu:22.04

# Alpine: ~7MB (10x smaller!)
docker pull alpine:3.19
```

4. **Clean Up Regularly**

```
# Remove unused images
docker image prune -a

# Remove unused volumes
docker volume prune

# Complete cleanup
docker system prune -a --volumes
```

Container Best Practices:

1. **Name Your Containers**

```
# Good - easy to identify
docker run --name web-server nginx

# Bad - random name
docker run nginx
```

2. **Use Restart Policies**

```
# Production containers
docker run --restart unless-stopped nginx
```

3. **Limit Resources**

```
docker run -m 512m --cpus="1" nginx
```

4. **Use Read-Only Filesystem When Possible**

```
docker run --read-only nginx
```

5. **Clean Up Temporary Containers**

```
# Auto-remove after stop
docker run --rm nginx
```

Chapter Summary

Key Takeaways:

- Images are read-only templates; containers are running instances
- Images consist of layers that are cached and reusable
- Proper image tagging is crucial for version management
- Container lifecycle: created → running → stopped → removed
- Port mapping connects host to container services
- Environment variables configure container behavior
- Volumes provide persistent data storage
- Resource limits prevent container resource exhaustion

Essential Commands:

```
# Images
docker pull <image>         # Download image
docker images               # List images
docker rmi <image>          # Remove image
docker tag <src> <dst>      # Tag image
docker history <image>      # View layers

# Containers
docker run <image>          # Create & start
docker ps                   # List running
docker ps -a                # List all
docker stop <container>     # Stop
docker rm <container>       # Remove
docker logs <container>     # View logs
docker exec -it <container> bash  # Access shell
```

Interview Questions - Chapter 3

Q1: Explain Docker image layers and why they matter.

Answer: Docker images are composed of multiple read-only layers stacked on top of each other. Each instruction in a Dockerfile creates a new layer. Benefits:

- **Caching**: Unchanged layers are reused, speeding up builds
- **Storage Efficiency**: Common layers are shared between images
- **Network Efficiency**: Only changed layers are pushed/pulled
- **Version Control**: Easy to track changes between image versions

Example: If you have 10 images based on `node:18-alpine`, that base layer is stored only once and shared across all images.

Q2: What's the difference between `docker run` and `docker start`?

Answer:

- `docker run`: Creates a NEW container from an image and starts it. Equivalent to `docker create` + `docker start`.
- `docker start`: Starts an existing STOPPED container.

```
# First time - creates new container
docker run --name web nginx

# After stopping
docker stop web

# Restart existing container
docker start web
```

Q3: How do port mappings work in Docker?

Answer: Port mapping connects a port on the host machine to a port in the container. Format: `-p host_port:container_port`

```
docker run -p 8080:80 nginx
# Host port 8080 → Container port 80
# Access via: localhost:8080
```

The container still listens on port 80 internally, but externally accessible via host's port 8080. Multiple containers can use the same internal port (80) as long as they map to different host ports (8080, 8081, etc.).

Q4: What happens to data when a container is removed?

Answer: By default, all data inside the container's writable layer is lost when the container is removed. To persist data:

1. **Named Volumes**: Managed by Docker

   ```
   docker run -v mydata:/app/data nginx
   ```

2. **Bind Mounts**: Link to host filesystem

   ```
   docker run -v C:\data:/app/data nginx
   ```

3. **tmpfs Mounts**: In-memory storage (lost on stop)

The volume data persists even after container removal.

Q5: Explain container restart policies and when to use each.

Answer:

Policy	Behavior	Use Case
`no`	Never restart (default)	One-time tasks, development
`on-failure`	Restart only on error	Batch jobs, task processors
`always`	Always restart, even after manual stop	System services
`unless-stopped`	Always restart unless manually stopped	Production services

```
# Production web server
docker run --restart unless-stopped nginx

# Development
docker run --restart no nginx

# Background job
docker run --restart on-failure myworker
```

Q6: How do you troubleshoot a container that keeps restarting?

Answer:

```
# 1. Check restart count
docker inspect <container> --format='{{.RestartCount}}'

# 2. View logs
docker logs <container>

# 3. Check exit code
docker inspect <container> --format='{{.State.ExitCode}}'

# 4. Run in foreground to see errors
docker run <image>

# 5. Check resource limits
docker stats <container>

# 6. Inspect configuration
docker inspect <container>
```

Common causes: application crash, misconfigured command, insufficient resources, missing dependencies, environment variable issues.

Chapter 4: Dockerfile Mastery

4.1 What is a Dockerfile?

A Dockerfile is a text file containing instructions to build a Docker image. It automates the image creation process and ensures reproducibility.

Analogy:
Think of a Dockerfile as a recipe. Just as a recipe has step-by-step instructions to make a dish, a Dockerfile has instructions to create an image.

Basic Structure:

```
# Dockerfile structure

FROM base_image           # Starting point
RUN command               # Execute commands
COPY source destination   # Copy files
WORKDIR /path            # Set working directory
ENV key=value            # Set environment variables
EXPOSE port              # Document ports
CMD ["executable"]       # Default command
```

4.2 Dockerfile Instructions

Complete Instruction Reference:

```
# ========== FROM ==========
# Base image - always first instruction
FROM node:18-alpine
FROM python:3.11-slim AS builder
FROM scratch                    # Empty image

# ========== LABEL ==========
# Metadata for image
LABEL maintainer="dev@example.com"
LABEL version="1.0"
LABEL description="My application"

# ========== ENV ==========
# Environment variables
ENV NODE_ENV=production
ENV PORT=3000
ENV DB_HOST=localhost \
    DB_PORT=5432 \
    DB_NAME=mydb

# ========== WORKDIR ==========
# Set working directory (creates if doesn't exist)
WORKDIR /app
WORKDIR /app/src

# ========== COPY ==========
# Copy files from host to image
COPY package.json .                     # Single file
COPY src/ /app/src/                     # Directory
COPY --chown=node:node app.js .         # With ownership

# ========== ADD ==========
# Like COPY but with extra features (auto-extract, URL support)
ADD archive.tar.gz /app/                # Auto-extracts
ADD https://example.com/file.txt /app/ # Download from URL
# Prefer COPY over ADD unless you need these features

# ========== RUN ==========
# Execute commands during build
RUN npm install                         # Simple command
RUN apt-get update && \                 # Multi-line with &&
    apt-get install -y curl && \
```

```
    rm -rf /var/lib/apt/lists/*

RUN ["npm", "install"]                # Exec form (no shell processing)

# ========== CMD ==========
# Default command when container starts (can be overridden)
CMD ["node", "server.js"]             # Exec form (preferred)
CMD node server.js                    # Shell form
CMD ["npm", "start"]

# ========== ENTRYPOINT ==========
# Fixed command when container starts (harder to override)
ENTRYPOINT ["python", "app.py"]
ENTRYPOINT ["docker-entrypoint.sh"]

# Combine ENTRYPOINT and CMD
ENTRYPOINT ["python"]
CMD ["app.py"]
# Result: python app.py
# Override: docker run myimage script.py → python script.py

# ========== EXPOSE ==========
# Document which ports container listens on (doesn't publish)
EXPOSE 3000
EXPOSE 80 443
EXPOSE 8080/tcp
EXPOSE 53/udp

# ========== VOLUME ==========
# Create mount point for persistent data
VOLUME /data
VOLUME ["/var/log", "/var/db"]

# ========== USER ==========
# Set user for subsequent instructions
USER node
USER 1000:1000
RUN whoami           # Runs as 'node' user

# ========== ARG ==========
# Build-time variables (only available during build)
ARG VERSION=1.0
ARG BUILD_DATE
```

```
ARG NODE_VERSION=18

RUN echo "Building version $VERSION"

# Build with: docker build --build-arg VERSION=2.0 .

# ========== ONBUILD ==========
# Trigger instructions when image is used as base
ONBUILD COPY . /app
ONBUILD RUN npm install

# ========== HEALTHCHECK ==========
# Check container health
HEALTHCHECK --interval=30s --timeout=3s --start-period=30s --retries=3 \
  CMD curl -f http://localhost/ || exit 1

HEALTHCHECK NONE  # Disable health check

# ========== SHELL ==========
# Override default shell
SHELL ["/bin/bash", "-c"]
SHELL ["powershell", "-command"]

# ========== STOPSIGNAL ==========
# Signal to stop container (default: SIGTERM)
STOPSIGNAL SIGTERM
STOPSIGNAL SIGKILL
```

4.3 Dockerfile Best Practices

1. Order Instructions by Change Frequency

```
# ❌ BAD - Reinstalls dependencies on every code change
FROM node:18-alpine
WORKDIR /app
COPY . .                    # Copies everything
RUN npm install             # Runs every time!

# ✅ GOOD - Caches dependencies
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./       # Only dependency files
RUN npm install             # Cached unless package.json changes
COPY . .                    # Code copied last
```

2. Minimize Layers

```
# ❌ BAD - Multiple layers
FROM ubuntu:22.04
RUN apt-get update
RUN apt-get install -y curl
RUN apt-get install -y vim
RUN apt-get install -y git

# ✅ GOOD - Single layer
FROM ubuntu:22.04
RUN apt-get update && \
    apt-get install -y curl vim git && \
    rm -rf /var/lib/apt/lists/*
```

3. Use .dockerignore

```
# .dockerignore file
node_modules
npm-debug.log
.git
.gitignore
.env
.DS_Store
*.md
Dockerfile
.dockerignore
dist/
build/
coverage/
*.log
```

4. Multi-Stage Builds

```
# ========== Stage 1: Build ==========
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# ========== Stage 2: Production ==========
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY package*.json ./
EXPOSE 3000
CMD ["node", "dist/server.js"]

# Result: Final image only contains production artifacts
```

5. Use Specific Tags

```
# ❌ BAD - Unpredictable
FROM node:latest

# ✅ GOOD - Specific version
FROM node:18.19.0-alpine

# ✅ BETTER - With digest for immutability
FROM node:18.19.0-alpine@sha256:abc123...
```

4.4 Real-World Dockerfile Examples

Example 1: Node.js Application

```
# Production-ready Node.js Dockerfile

# Build stage
FROM node:18-alpine AS builder

# Set working directory
WORKDIR /app

# Copy dependency files
COPY package*.json ./

# Install dependencies
RUN npm ci --only=production && \
    npm cache clean --force

# Copy application code
COPY . .

# Build application (if needed)
RUN npm run build

# Production stage
FROM node:18-alpine

# Set NODE_ENV
ENV NODE_ENV=production

# Create non-root user
RUN addgroup -g 1001 -S nodejs && \
    adduser -S nodejs -u 1001

# Set working directory
WORKDIR /app

# Copy dependencies and built app from builder
COPY --from=builder --chown=nodejs:nodejs /app/node_modules ./node_modules
COPY --from=builder --chown=nodejs:nodejs /app/dist ./dist
COPY --from=builder --chown=nodejs:nodejs /app/package*.json ./

# Switch to non-root user
USER nodejs
```

```
# Expose port
EXPOSE 3000

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD node -e "require('http').get('http://localhost:3000/health', (r) =>
{process.exit(r.statusCode === 200 ? 0 : 1)})"

# Start application
CMD ["node", "dist/server.js"]
```

Example 2: React Application

```
# Multi-stage build for React app

# Build stage
FROM node:18-alpine AS builder

WORKDIR /app

# Install dependencies
COPY package*.json ./
RUN npm ci

# Copy source and build
COPY . .
RUN npm run build

# Production stage - Serve with nginx
FROM nginx:1.25.3-alpine

# Copy built files to nginx
COPY --from=builder /app/build /usr/share/nginx/html

# Copy custom nginx configuration (optional)
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Expose port
EXPOSE 80

# Start nginx
CMD ["nginx", "-g", "daemon off;"]
```

Example 3: Python Flask Application

```
# Python Flask Dockerfile

FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Set environment variables
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

# Install system dependencies
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        gcc \
        postgresql-client && \
    rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application
COPY . .

# Create non-root user
RUN useradd -m -u 1000 appuser && \
    chown -R appuser:appuser /app
USER appuser

# Expose port
EXPOSE 5000

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:5000/health || exit 1

# Run application
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "4", "app:app"]
```

Example 4: Next.js Application

```
# Next.js production Dockerfile

FROM node:18-alpine AS base

# Dependencies stage
FROM base AS deps
RUN apk add --no-cache libc6-compat
WORKDIR /app

COPY package*.json ./
RUN npm ci

# Builder stage
FROM base AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .

ENV NEXT_TELEMETRY_DISABLED 1

RUN npm run build

# Runner stage
FROM base AS runner
WORKDIR /app

ENV NODE_ENV production
ENV NEXT_TELEMETRY_DISABLED 1

RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 nextjs

COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./.next/static

USER nextjs

EXPOSE 3000

ENV PORT 3000
```

```
CMD ["node", "server.js"]
```

Example 5: Microservice with Health Checks

```
# Microservice Dockerfile with advanced features

FROM golang:1.21-alpine AS builder

WORKDIR /build

# Copy dependency files
COPY go.mod go.sum ./
RUN go mod download

# Copy source code
COPY . .

# Build binary
RUN CGO_ENABLED=0 GOOS=linux go build -a -installsuffix cgo -o app .

# Final stage
FROM alpine:3.19

# Install CA certificates for HTTPS
RUN apk --no-cache add ca-certificates

WORKDIR /root/

# Copy binary from builder
COPY --from=builder /build/app .

# Add health check script
COPY healthcheck.sh .
RUN chmod +x healthcheck.sh

# Expose port
EXPOSE 8080

# Health check
HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
  CMD /root/healthcheck.sh

# Run application
ENTRYPOINT ["./app"]
```

4.5 Building Images

Basic Build Commands:

```
# ========== BASIC BUILD ==========
# Build from Dockerfile in current directory
docker build -t myapp:v1 .

# Build from specific Dockerfile
docker build -t myapp:v1 -f Dockerfile.prod .

# Build with no cache
docker build --no-cache -t myapp:v1 .

# ========== BUILD WITH TAGS ==========
# Multiple tags
docker build -t myapp:v1 -t myapp:latest .

# With registry
docker build -t myregistry.com/myapp:v1 .

# ========== BUILD ARGUMENTS ==========
# Pass build arguments
docker build --build-arg VERSION=1.0 --build-arg ENV=prod -t myapp .

# Example Dockerfile using ARG:
# ARG VERSION
# LABEL version=$VERSION

# ========== MULTI-PLATFORM BUILD ==========
# Build for multiple architectures
docker buildx build --platform linux/amd/64,linux/arm64 -t myapp:v1 .

# ========== BUILD WITH TARGET ==========
# Build specific stage in multi-stage Dockerfile
docker build --target builder -t myapp:builder .

# ========== BUILD CONTEXT ==========
# Build from Git repository
docker build -t myapp https://github.com/user/repo.git#main

# Build from tar archive
docker build -t myapp - < context.tar.gz

# ========== VIEWING BUILD OUTPUT ==========
# Show detailed build output
```

```
docker build --progress=plain -t myapp .

# Save build output
docker build -t myapp . 2>&1 | Tee-Object build.log
```

BuildKit Features:

```
# Enable BuildKit (faster builds)
$env:DOCKER_BUILDKIT=1

# Build with BuildKit
docker build -t myapp .

# BuildKit with cache mount (faster dependency installs)
# In Dockerfile:
# RUN --mount=type=cache,target=/root/.npm npm install
```

Lab 4.1: Creating Your First Dockerfile

Objective: Build a custom image for a simple web application.

Task 1: Create Simple Node.js App

```
# Create project directory
New-Item -ItemType Directory -Path C:\Projects\docker-lab4
Set-Location C:\Projects\docker-lab4

# Create package.json
@'
{
  "name": "docker-demo",
  "version": "1.0.0",
  "main": "server.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
'@ | Out-File -FilePath package.json

# Create server.js
@'
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.json({
    message: 'Hello from Docker!',
    timestamp: new Date().toISOString(),
    environment: process.env.NODE_ENV || 'development'
  });
});

app.get('/health', (req, res) => {
  res.status(200).json({ status: 'healthy' });
});

app.listen(PORT, '0.0.0.0', () => {
  console.log(`Server running on port ${PORT}`);
});
'@ | Out-File -FilePath server.js
```

Task 2: Create Dockerfile

```
# Dockerfile
FROM node:18-alpine

# Set working directory
WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm install --production

# Copy application files
COPY server.js .

# Set environment variable
ENV NODE_ENV=production
ENV PORT=3000

# Expose port
EXPOSE 3000

# Health check
HEALTHCHECK --interval=30s --timeout=3s CMD node -e
"require('http').get('http://localhost:3000/health', (r) =>
{process.exit(r.statusCode === 200 ? 0 : 1)})"

# Start application
CMD ["node", "server.js"]
```

Task 3: Create .dockerignore

```
# .dockerignore
node_modules
npm-debug.log
.git
.gitignore
*.md
Dockerfile
.dockerignore
.env
```

Task 4: Build and Run

```
# Build image
docker build -t mywebapp:v1 .

# Verify image
docker images mywebapp

# Run container
docker run -d -p 3000:3000 --name webapp mywebapp:v1

# Test application
curl http://localhost:3000
curl http://localhost:3000/health

# View logs
docker logs webapp

# Check health status
docker inspect webapp --format='{{.State.Health.Status}}'

# Clean up
docker stop webapp
docker rm webapp
```

Lab 4.2: Multi-Stage Builds

Objective: Create optimized images using multi-stage builds.

Task 1: Create React Application

```
# Create React app directory
New-Item -ItemType Directory -Path C:\Projects\react-docker
Set-Location C:\Projects\react-docker

# Create package.json
@'
{
  "name": "react-docker-app",
  "version": "1.0.0",
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build"
  },
  "devDependencies": {
    "react-scripts": "^5.0.1"
  }
}
'@ | Out-File -FilePath package.json

# Create public/index.html
New-Item -ItemType Directory -Path public
@'
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>React Docker App</title>
</head>
<body>
  <div id="root"></div>
</body>
</html>
'@ | Out-File -FilePath public\index.html

# Create src/index.js
New-Item -ItemType Directory -Path src
@'
import React from 'react';
```

```
import ReactDOM from 'react-dom/client';

function App() {
  return (
    <div style={{textAlign: 'center', marginTop: '50px'}}>
      <h1>Hello from React Docker!</h1>
      <p>Built with multi-stage Docker build</p>
    </div>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
'@ | Out-File -FilePath src\index.js
```

Task 2: Create Multi-Stage Dockerfile

```
# Dockerfile.multistage
# Stage 1: Build React app
FROM node:18 AS builder

WORKDIR /app

# Install dependencies
COPY package*.json ./
RUN npm install

# Copy source and build
COPY public ./public
COPY src ./src
RUN npm run build

# Stage 2: Serve with Nginx
FROM nginx:1.25.3-alpine

# Copy built files from builder stage
COPY --from=builder /app/build /usr/share/nginx/html

# Copy custom nginx config (optional)
RUN echo 'server { \
    listen 80; \
    location / { \
        root /usr/share/nginx/html; \
        index index.html; \
        try_files $uri $uri/ /index.html; \
    } \
}' > /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Task 3: Build and Compare

```
# Build multi-stage image
docker build -f Dockerfile.multistage -t react-app:multistage .

# For comparison, build single-stage image
@'
FROM node:18
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY public ./public
COPY src ./src
RUN npm run build
RUN npm install -g serve
EXPOSE 3000
CMD ["serve", "-s", "build", "-l", "3000"]
'@ | Out-File -FilePath Dockerfile.single

docker build -f Dockerfile.single -t react-app:single .

# Compare image sizes
docker images react-app

# Expected output:
# react-app:multistage  ~  40MB  (nginx)
# react-app:single      ~ 1.2GB (includes Node.js + all dependencies)

# Run multi-stage version
docker run -d -p 8080:80 --name react-multi react-app:multistage

# Test in browser: http://localhost:8080

# Clean up
docker stop react-multi
docker rm react-multi
```

Lab 4.3: Advanced Dockerfile Techniques

Objective: Implement advanced Dockerfile patterns.

Task 1: Build Arguments and Environment Variables

```
# Dockerfile.args
ARG NODE_VERSION=18
ARG APP_PORT=3000

FROM node:${NODE_VERSION}-alpine

# Build arguments available during build
ARG BUILD_DATE
ARG VERSION
ARG GIT_COMMIT

# Convert to environment variables for runtime
ENV APP_VERSION=${VERSION}
ENV BUILD_DATE=${BUILD_DATE}
ENV GIT_COMMIT=${GIT_COMMIT}
ENV PORT=${APP_PORT}

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

# Add labels
LABEL version="${VERSION}"
LABEL build-date="${BUILD_DATE}"
LABEL git-commit="${GIT_COMMIT}"

EXPOSE ${APP_PORT}

CMD ["node", "server.js"]
```

Build with arguments:

```
docker build \
  --build-arg NODE_VERSION=18 \
  --build-arg VERSION=1.0.0 \
  --build-arg BUILD_DATE=$(Get-Date -Format "yyyy-MM-dd") \
  --build-arg GIT_COMMIT=abc123 \
  --build-arg APP_PORT=3000 \
  -t myapp:args \
  -f Dockerfile.args \
  .

# Inspect labels
docker inspect myapp:args --format='{{json .Config.Labels}}' | ConvertFrom-Json
```

Task 2: Health Checks

```
# Dockerfile.health
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

# Install curl for health check
RUN apk add --no-cache curl

# Health check with custom interval
HEALTHCHECK --interval=10s \
            --timeout=3s \
            --start-period=30s \
            --retries=3 \
  CMD curl -f http://localhost:3000/health || exit 1

EXPOSE 3000

CMD ["node", "server.js"]
```

```
# Build and run
docker build -t myapp:health -f Dockerfile.health .
docker run -d -p 3000:3000 --name health-app myapp:health

# Watch health status
docker ps
# Check "STATUS" column - should show "healthy" after 30s

# View health check history
docker inspect health-app --format='{{json .State.Health}}' | ConvertFrom-Json |
ConvertTo-Json
```

Task 3: Non-Root User

```
# Dockerfile.secure
FROM node:18-alpine

# Create app directory
WORKDIR /app

# Create non-root user
RUN addgroup -g 1001 -S nodejs && \
    adduser -S nodejs -u 1001

# Copy with root permissions
COPY package*.json ./
RUN npm install

# Copy application files
COPY . .

# Change ownership to nodejs user
RUN chown -R nodejs:nodejs /app

# Switch to non-root user
USER nodejs

EXPOSE 3000

CMD ["node", "server.js"]
```

```
# Build and verify
docker build -t myapp:secure -f Dockerfile.secure .
docker run -d -p 3000:3000 --name secure-app myapp:secure

# Verify running as non-root user
docker exec secure-app whoami
# Output: nodejs

# Try to write to protected area (should fail)
docker exec secure-app touch /etc/test
# Error: cannot create /etc/test: Permission denied
```

Troubleshooting Dockerfile Issues

Issue 1: Build fails at specific layer

```
# Build with verbose output
docker build --progress=plain -t myapp .

# Use --no-cache to avoid cached layers
docker build --no-cache -t myapp .

# Build up to specific stage
docker build --target builder -t myapp:debug .
```

Issue 2: Large image size

```
# Check layer sizes
docker history myapp:v1

# Use dive tool for detailed analysis
docker run --rm -it `
  -v /var/run/docker.sock:/var/run/docker.sock `
  wagoodman/dive:latest myapp:v1

# Solutions:
# 1. Use alpine base images
# 2. Use multi-stage builds
# 3. Clean up in same RUN command
# 4. Remove unnecessary files
```

Issue 3: COPY doesn't work

```
# Verify build context
docker build -t myapp . 2>&1 | Select-String "COPY"

# Check .dockerignore isn't excluding files
Get-Content .dockerignore

# Use absolute paths in Dockerfile
COPY ./src /app/src
```

Issue 4: Environment variables not working

```
# Wrong - runtime value
RUN echo $MY_VAR  # Won't work

# Right - build-time ARG
ARG MY_VAR
RUN echo $MY_VAR

# Right - runtime ENV
ENV MY_VAR=value
```

Best Practices Summary

Image Size Optimization:

1. Use minimal base images (alpine)
2. Multi-stage builds for compiled languages
3. Combine RUN commands
4. Clean up in same layer
5. Use .dockerignore

Security:

1. Don't run as root
2. Scan images for vulnerabilities
3. Use specific image versions
4. Don't include secrets in image
5. Minimize installed packages

Build Performance:

1. Order instructions by change frequency
2. Leverage layer caching
3. Use BuildKit
4. Minimize build context
5. Use .dockerignore

Maintainability:

1. Use labels for metadata
2. Document with comments
3. Use health checks
4. Pin dependency versions
5. Follow consistent naming

Chapter Summary

Key Takeaways:

- Dockerfiles automate image creation
- Layer caching speeds up builds
- Multi-stage builds create smaller images
- Order instructions strategically
- Use .dockerignore to exclude files
- Run containers as non-root users
- Implement health checks for reliability
- ARG for build-time, ENV for runtime variables

Essential Dockerfile Instructions:

```
FROM <image>           # Base image
WORKDIR /path          # Set working directory
COPY <src> <dest>      # Copy files
RUN <command>          # Execute commands
ENV KEY=value          # Environment variables
EXPOSE <port>          # Document ports
CMD ["executable"]     # Default command
HEALTHCHECK CMD        # Health check command
USER <user>            # Set user
```

Build Command Patterns:

```
# Basic build
docker build -t name:tag .

# With build arguments
docker build --build-arg KEY=value -t name:tag .

# Multi-stage specific target
docker build --target builder -t name:tag .

# No cache
docker build --no-cache -t name:tag .
```

Interview Questions - Chapter 4

Q1: Explain the difference between CMD and ENTRYPOINT.

Answer:

- **CMD**: Provides default command/arguments. Can be overridden by providing arguments to `docker run`.
- **ENTRYPOINT**: Defines the fixed executable. Arguments from `docker run` are appended to it.

```
# CMD only
CMD ["python", "app.py"]
# docker run myimage script.py  → Runs: script.py (CMD replaced)

# ENTRYPOINT only
ENTRYPOINT ["python"]
# docker run myimage script.py  → Runs: python script.py

# Both (preferred pattern)
ENTRYPOINT ["python"]
CMD ["app.py"]
# docker run myimage           → Runs: python app.py
# docker run myimage script.py → Runs: python script.py
```

Q2: What are multi-stage builds and why are they important?

Answer: Multi-stage builds use multiple FROM statements to create intermediate build stages. The final image only contains artifacts copied from previous stages, not build dependencies.

Benefits:

- **Smaller images**: Final image doesn't include build tools
- **Security**: Fewer tools mean smaller attack surface
- **Clarity**: Separate build and runtime concerns
- **Speed**: Parallel stage execution

Example: Build stage has compilers and dev tools (1GB+), production stage has only compiled binary (~50MB).

Q3: How does layer caching work and how can you optimize it?

Answer: Docker caches each layer. If an instruction and its context haven't changed, Docker reuses the cached layer instead of rebuilding.

Optimization strategies:

```
# Bad - cache busted on any code change
COPY . .
RUN npm install

# Good - cache dependencies separately
COPY package*.json ./
RUN npm install
COPY . .
```

Order instructions from least to most frequently changing. Combine related commands to reduce layers.

Q4: What is .dockerignore and why should you use it?

Answer: .dockerignore excludes files from the build context, similar to .gitignore.

Benefits:

- **Faster builds**: Smaller context means faster upload to daemon
- **Smaller images**: Prevents copying unnecessary files
- **Security**: Prevents accidental inclusion of secrets (.env files)
- **Cleaner builds**: Excludes git history, logs, local config

Common entries:

```
node_modules
.git
*.log
.env
Dockerfile
README.md
```

Q5: Explain ARG vs ENV in Dockerfiles.

Answer:

Feature	ARG	ENV
Scope	Build-time only	Build + runtime
Override	`--build-arg`	`docker run -e`
In image	No (unless converted to ENV)	Yes
Use case	Build configuration	Runtime configuration

```
ARG NODE_VERSION=18      # Build-time
FROM node:${NODE_VERSION}

ENV NODE_ENV=production  # Runtime
RUN echo $NODE_ENV       # Available in build
CMD node app.js          # Available in container
```

Q6: How do you create secure Docker images?

Answer:

1. **Use official base images**
2. **Run as non-root user**

   ```
   USER nodejs
   ```

3. **Scan for vulnerabilities**

   ```
   docker scan myimage
   ```

4. **Use specific versions**

   ```
   FROM node:18.19.0-alpine
   ```

5. **Don't include secrets**

```
# Bad
ENV API_KEY=secret

# Good - pass at runtime
docker run -e API_KEY=secret myimage
```

6. **Minimize installed packages**
7. **Use read-only filesystem**

```
docker run --read-only myimage
```

8. **Implement health checks**

Due to length constraints, I've provided the first 4 comprehensive chapters of this Docker & Kubernetes guide. This represents approximately 20% of the complete guide.

What Has Been Covered:

Part 1: Docker Fundamentals (Chapters 1-4)

✅ **Chapter 1: Introduction to Containerization**

- Container concepts and benefits
- VMs vs Containers comparison
- Container ecosystem overview
- Real-world use cases

✅ **Chapter 2: Docker Architecture & Installation**

- Docker architecture deep dive
- Windows-specific installation (WSL 2 / Hyper-V)
- Docker Desktop configuration
- CLI basics and verification

✅ **Chapter 3: Docker Images & Containers**

- Image layers and caching
- Complete image operations
- Container lifecycle management
- Advanced container scenarios

✅ **Chapter 4: Dockerfile Mastery**

- All Dockerfile instructions
- Multi-stage builds
- Best practices and optimization
- Real-world examples (Node.js, React, Python, Go)

Remaining Content Structure:

Part 1: Docker (Continued)

- **Chapter 5: Docker Networking** - Bridge, host, overlay networks, container communication
- **Chapter 6: Docker Volumes** - Data persistence, volume types, backup strategies
- **Chapter 7: Docker Compose** - Multi-container applications, YAML syntax, real projects
- **Chapter 8: Advanced Docker** - Security, registries, optimization, production practices

Part 2: Kubernetes Fundamentals

- **Chapter 9-16**: Kubernetes introduction, architecture, Minikube setup, Pods, Services, ConfigMaps, Secrets, Storage, Deployments

Part 3: Advanced Kubernetes

- **Chapter 17-22**: StatefulSets, Ingress, Resource Management, Health Checks, Helm, CI/CD

Part 4: Real-World Projects

- **Chapter 23-25**: Complete containerized applications (React, Python microservices, full-stack)

Appendices

- Command references, troubleshooting, best practices checklists

Each remaining chapter will follow the same structure:

- Theory with diagrams
- Step-by-step labs
- Troubleshooting sections
- Best practices
- Interview questions
- Chapter summaries

Would you like me to:

1. **Continue with the next chapters** (5-8 for Docker, then Kubernetes sections)?
2. **Focus on a specific chapter** you'd like to see completed?
3. **Export current content** in a format ready for PDF conversion?
4. **Add more detail** to any existing chapter?